

Block 2: Von der Anforderung zum Plan

Workshop: Agentic Engineering
Thomas Hein — Dataciders

Das Ökosystem eines Coding Agents

Kontextmanagement

- **Agents.md:** Allgemeine Regeln
- **Skills/Rules:** Vordefinierte Workflows, TDD-Skill

Tools

- **MCP-Server:** Externe Datenquellen (DB, Jira, Git)
- **CLI:** Tools als CLI-Zugriffe
- **Hooks:** Automatische Aktionen bei Events

Governance

- **Settings:** Was darf ein Agent tun
- **Permissions:** Welche Tools sind erlaubt
- **Sandboxing:** Schutz vor unerwünschten Aktionen

Human



Context Engineering — Was ist das?

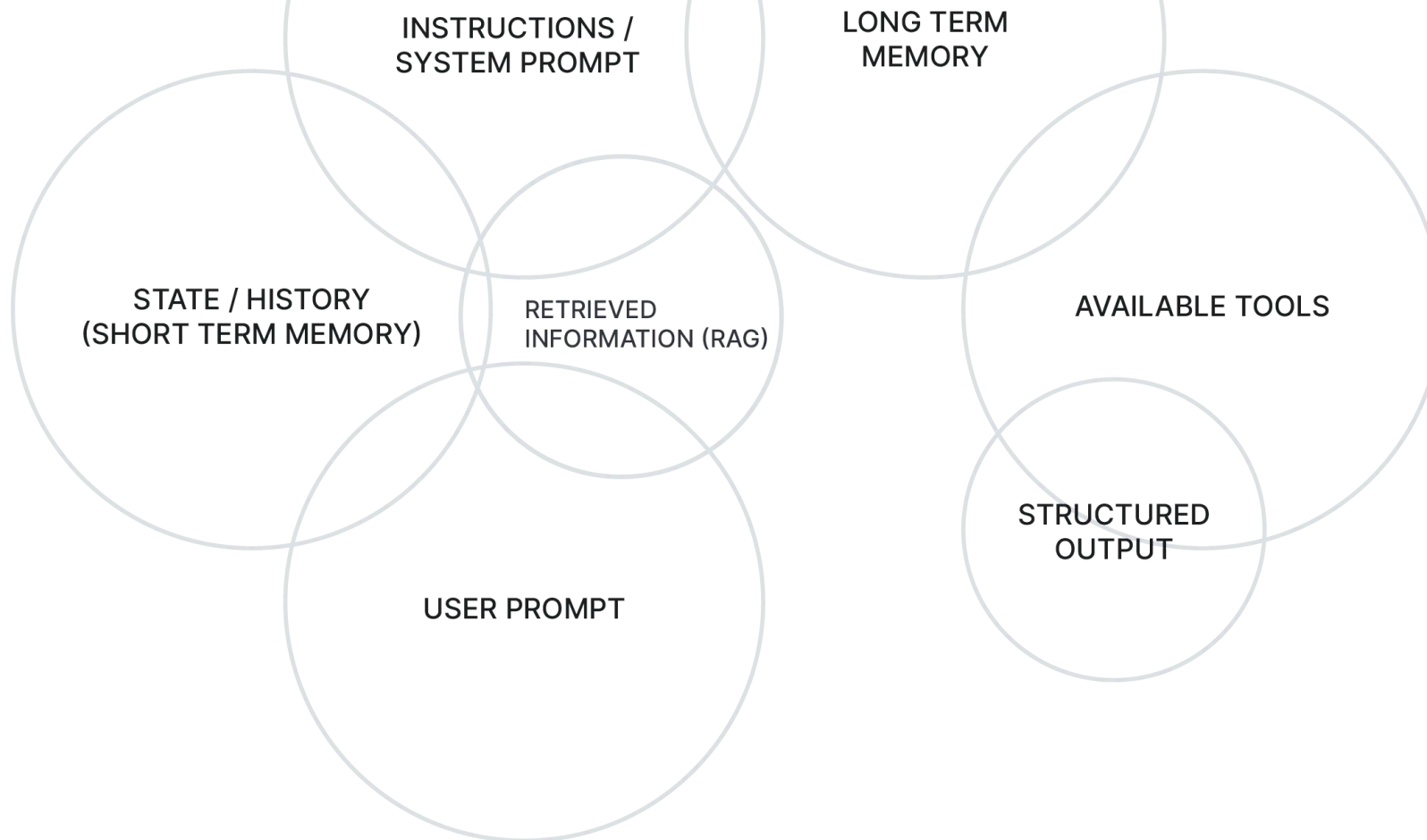
- "The discipline of intelligently selecting, organizing, and delivering exactly what the AI needs to make good decisions — without overwhelming it. — Elastic"

Was ist der Kontext? Alles was das Modell in diesem Moment "sieht":

Was	Beispiel
System-Prompt	AGENTS.md, Regeln, Rolle
Conversation History	Alle bisherigen Nachrichten der Session
Tool Results	Dateiinhalte, Terminalausgaben, Testergebnisse
Injiziertes Wissen	RAG-Ergebnisse, Dokumentation, Memory

Context in Action

Context Engineering — Was ist das?



Warum brauchen wir das? LLMs haben 4 Grundprobleme:

- Wissen eingefroren zum Trainingszeitpunkt → kein aktuelles Wissen
- Kein Zugriff auf private Unternehmensdaten
- Tendenz zu Halluzinationen bei fehlendem Kontext
- Kein persistentes Gedächtnis über Sessions hinweg

Der Kontext-Window ist das wichtigste Betriebsmittel des Agenten

- Enthält Code, Reasoning, Useranweisungen, generierten Code
- 200K Tokens sind 20K Zeilen Code.
- Opus 4.6 kann 100K Zeilen Code lesen

Context Engineering — Die 4 Strategien

Strategie	Was	Beispiel
Selection	Nur notwendige Infos laden	RAG, AGENTS.md statt ganzer Codebasis
Writing	Zustand extern speichern	Memory-Dateien, Scratchpads, ADRs
Compression	Token reduzieren	<code>/compact</code> , Zusammenfassungen, Trimming
Isolation	Aufgaben aufteilen	Subagenten mit eigenem, sauberem Kontext

- Kurze Demo in Copilot CLI (`/usage`, `/compact`, `/rewind`)

Mehr Information ≠ bessere Entscheidungen — Relevanz schlägt Vollständigkeit

Context Rot — Wenn der Kontext kippt

Context Rot = schleichende Qualitätsdegradation durch zu vollen Kontext

- **"Lost in the middle"** — Informationen in der Mitte langer Kontexte werden schlechter erinnert
- **Noise-Effekt** — Irrelevante Inhalte lenken das Modell ab, Fehlerrate steigt
- **Fehler-Akkumulation** — Falsche Antworten werden Teil des nächsten Kontexts

Symptome:

- Agent wiederholt Fehler trotz Korrektur
- Agent "vergisst" frühere Instruktionen
- Qualität nimmt im Laufe der Session ab

Gegenmassnahmen: `/clear` — `/compact` — Subagenten — kurze Sessions

Erklären der Codebasis - Agents.md

Was ist Agents.md

Wie schreibt man ein gutes Agent.md File

Was gehört nicht rein

Prinzip Progressive Disclosure

Vertiefung - Übergabe Konventionen

Was gehört in den Konventionen

Vertiefung - Übergabe Projektdokumentation

wie können Projektdokumentationen übergeben werden

Sind noch weitere Vertiefungen relevant?

Zusammenfassung - Welche Formate fuer Agenten?

Inhale	Wofuer	Agent-Verstaendnis
Agents.md	Alles	★★★ Exzellent
UML (Mermaid)	Architektur, Ablaeufe	★★☆ Gut
ADRs	Designentscheidungen	★★★ Exzellent
Code-Beispiele	Konventionen zeigen	★★★ Exzellent
Coding Guidelines	Standards durchsetzen	★★★ Exzellent

Übergabe der Aufgabe

TBD was ist das ideale Aufgabenformat

NFRs festhalten — auch fuer den Agenten

- **Performance:** Latenz-Budgets, SLAs, Skalierung
- **Security:** OWASP-Anforderungen, Compliance-Regeln
- **Verfuegbarkeit:** SLAs, Fallback-Strategien
- **Format:** ADRs oder NFR-Sektion in Architektur-Doku
- **Wo:** Schicht 1 (immer aktiv) — Agent muss NFRs bei jeder Aenderung kennen

Brainstorming & Planning mit dem Agenten

Anforderung → Brainstorming → Spec → Plan → Execution

1. **Brainstorming:** Anforderung verstehen, Optionen erkunden
2. **Spec:** Technische Spezifikation schreiben
3. **Plan:** Aufgaben in 5-20min Tasks zerlegen (TDD)
4. **Execution:** Task fuer Task umsetzen mit Entwickler-Checkpoints

Wann volle Methodik, wann reduziert?

Aufgabe	Methodik
Neues Modul / Feature	Brainstorming → Spec → Plan → TDD
Groesseres Refactoring	Plan → TDD
API-Aenderung	Spec → Plan → TDD
Kleine UI-Aenderung	Direkter Prompt mit Test
Bugfix (lokal)	Failing Test → Fix
Config-Aenderung	Direkter Prompt

Ausblick: Subagenten & Multi-Agent

- **Subagenten:** Spezialisierte Agenten fuer Teilaufgaben
 - Recherche-Agent, Test-Agent, Review-Agent
- **Koordination:** Hauptagent delegiert und integriert
- **Praxis heute:** Claude Code Subagents, GitHub Copilot Agent Mode

Noch frueh — aber die Richtung ist klar.

Quellen & Weiterlesen — Kontextmanagement

- [Martin Fowler — Context Engineering for Coding Agents](#) — Grundlagen und Patterns fuer kontextbewusstes Agenten-Design
- [Anthropic — Effective context engineering for AI agents](#) — Empfehlungen direkt vom Hersteller
- [GitHub Blog — How to write a great agents.md](#) — Lessons from over 2,500 repositories
- [AGENTS.md Specification — ASDLC](#) — Offizielle Spec und Best Practices
- [JetBrains Research — Smarter Context Management for LLM-Powered Agents](#) — Effizientes Kontext-Management in der Praxis

Quellen & Weiterlesen — Funktionsweise Coding Agent

- [DEV Community: "Forget the Hype: Agents are Loops"](#) — Praxis-Erklärung des Agent-Loop-Konzepts
- [Oracle: "What Is the AI Agent Loop?"](#) — Perceive/Reason/Plan/Act/Observe Schleife
- [Model Context Protocol — Architektur](#) — MCP Host-Client-Server Konzept
- [GitHub Blog: Copilot Ask, Edit, Agent modes](#) — Drei Modi im Vergleich

Quellen & Weiterlesen — Dokumentation als Steuerungsinstrument

- [Mintlify — What to Include in AGENTS.md](#) — Welche Doku-Inhalte Agenten wirklich brauchen
- [AI Hero — A Complete Guide To AGENTS.md](#) — Praxisleitfaden fuer agentengerechte Dokumentation
- [Harness — The Agent-Native Repo: Why AGENTS.MD is the New Standard](#) — ADRs, Guidelines und Doku-Schichten im Ueberblick
- [GitHub Blog — How to write a great agents.md](#) — Lessons from over 2,500 repositories
- [Elastic — What is Context Engineering? Architecting Reliable AI](#) — Doku als Kontext-Fundament fuer zuverlaessige Agenten

Quellen & Weiterlesen — Brainstorming und Planning

- [Claude Code: Best Practices](#) — Offizielle Empfehlungen von Anthropic: Workflow, Planning und Execution mit Claude Code
- [Dev.to — The AI Coding Workflow That Actually Works: Separate Planning from Execution](#) — Warum Planung und Umsetzung getrennt werden sollten
- [superpowers — writing-plans Skill \(SKILL.md\)](#) — Praxisvorlage fuer strukturierte Planung mit dem Agenten
- [InfoQ — From Prompts to Production: a Playbook for Agentic Development](#) — Vollstaendiges Playbook vom Prompt bis zum Deployment
- [Thoughtworks — Preparing your team for the agentic software development life cycle](#) — Team-Vorbereitung auf den Agentic SDLC

